CprE 3810: Computer Organization and Assembly-Level Programming

Project Part 2 Report

Team Members: Shawn Robinson

Jay Patel

Project Teams Group #: IS_01

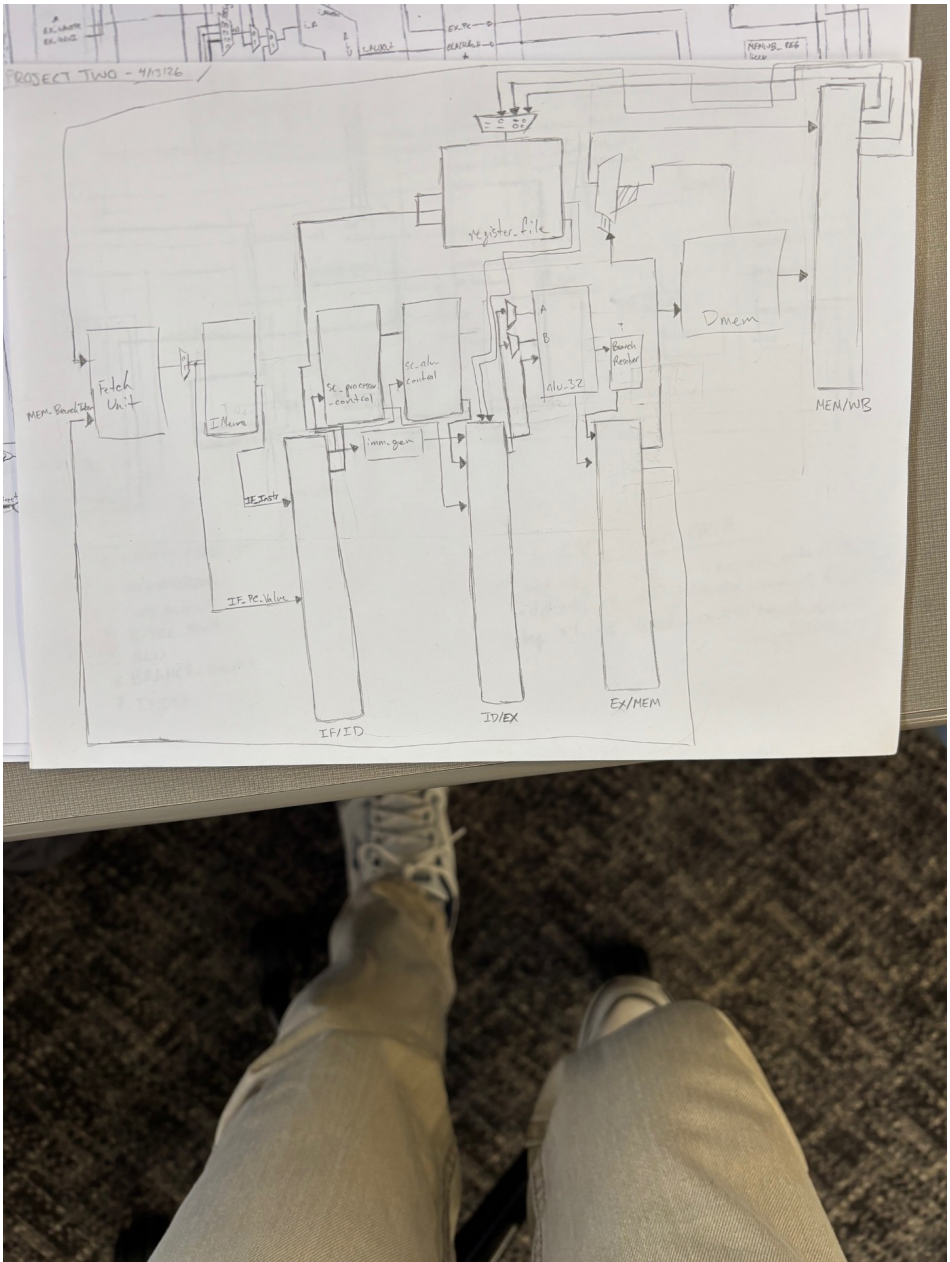
Refer to the highlighted language in the project 1 instruction for the context of the following questions.

[1.a] Come up with a global list of the datapath values and control signals that are required during each pipeline stage.

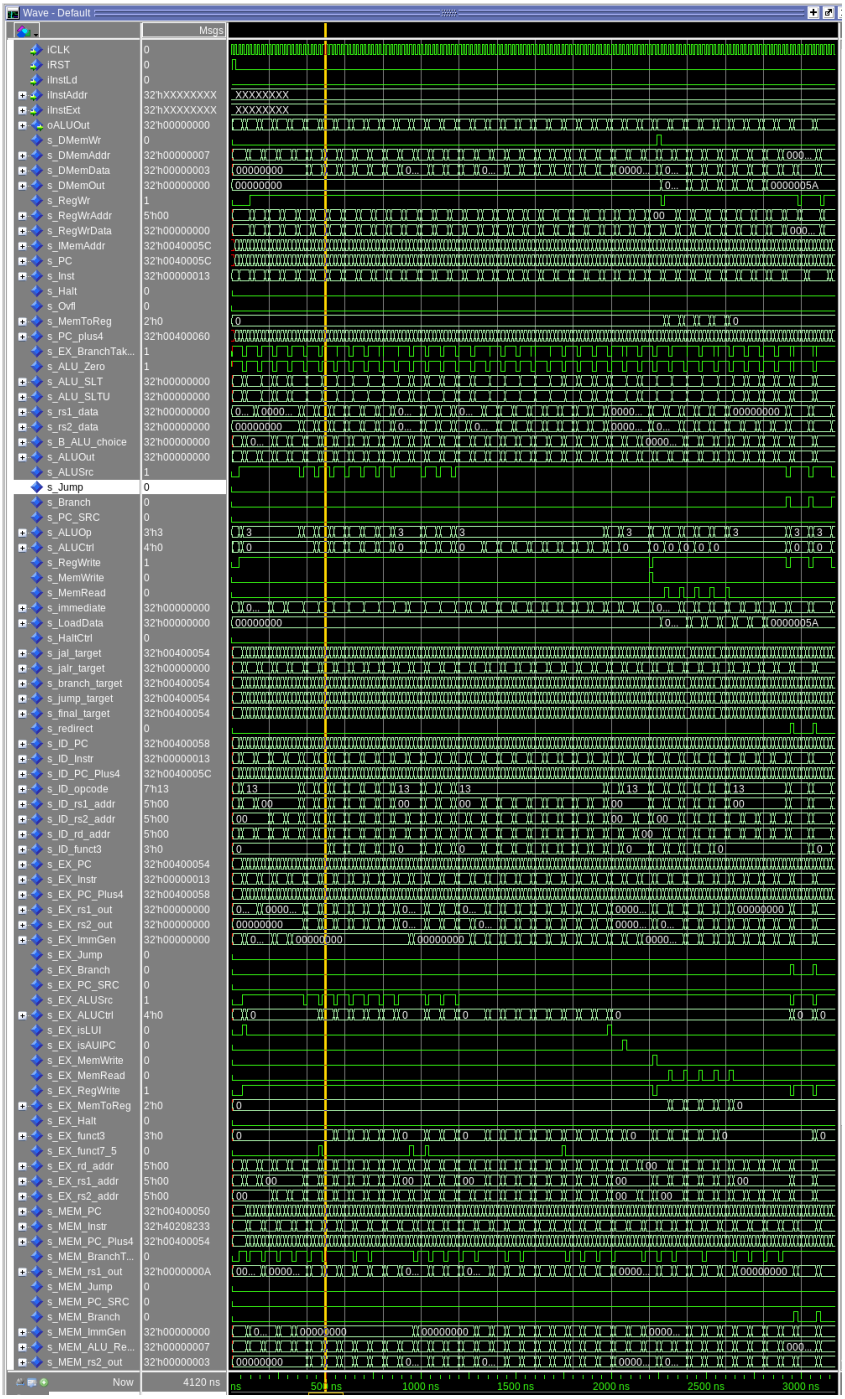
IF	PC
	Instr
	PC Plus 4
ID	PC
	Instr
	PC Plus 4
EX	PC
	Instr
	PC Plus 4
	rs1
	rs2
	immediate generated
	[CONTROLS]
	ALUSrc
	ALUCtrl
	isLUI
	isAUIPC
	Jump
	Branch
	PC SRC
	MemWrite
	MemRead
	RegWrite
	MemToReg
MEM	PC
	Instr
	PC Plus 4
	rs2
	ALU Result
	Jump
	Branch
	PC SRC
	MemWrite
	MemRead
	RegWrite

WB	MemToReg
	PC
	Instr
	PC Plus 4
	ALU Result
	DMemOut
	Rd Addr
	Halt

[1.b.ii] high-level schematic drawing of the interconnection between components.



[1.c.i] include an annotated waveform in your writeup and provide a short discussion of result correctness.

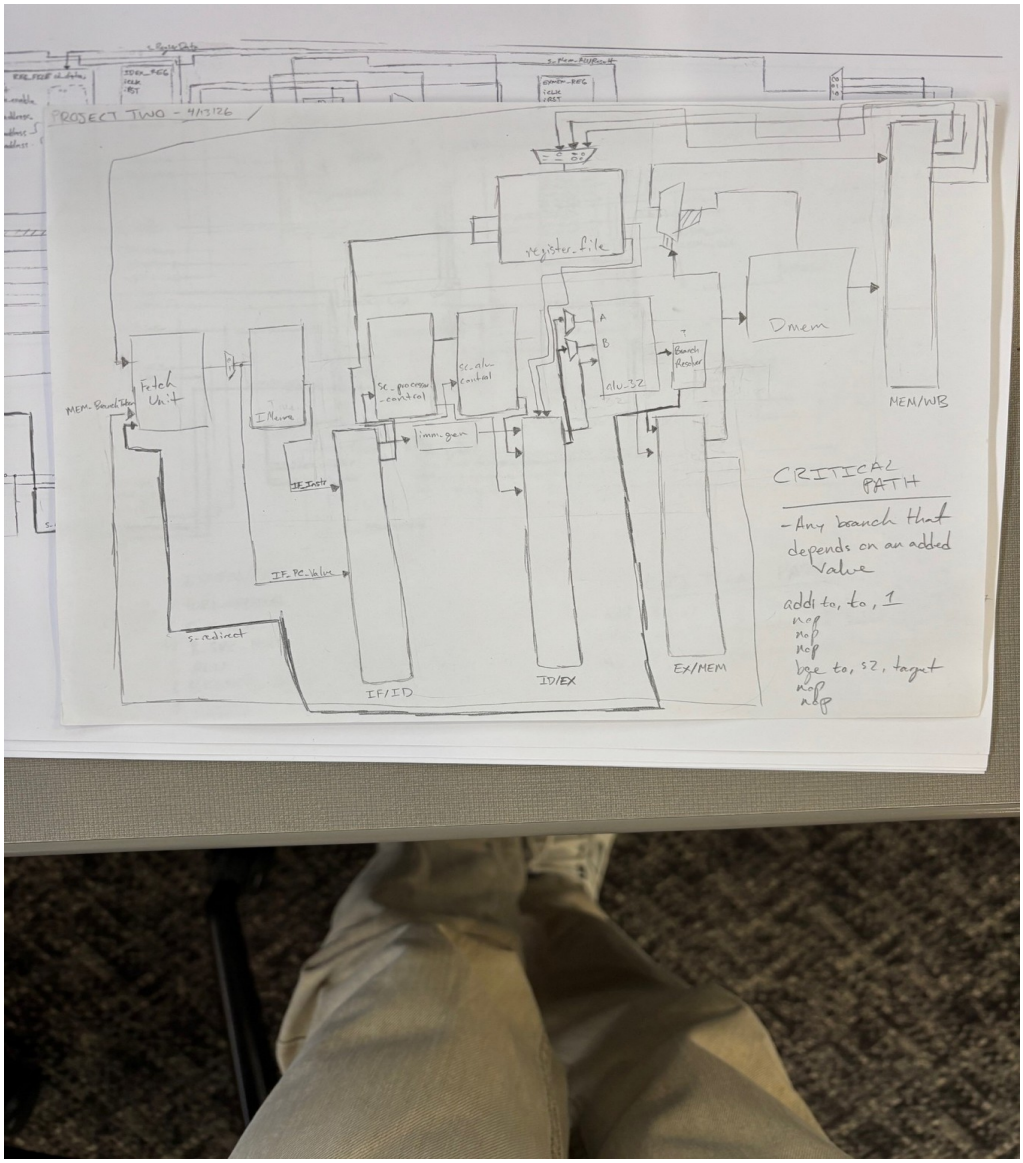


[1.c.ii] Include an annotated waveform in your writeup of two iterations or recursions of these programs executing correctly and provide a short discussion of result correctness. In your waveform and annotation, provide 3 different examples (at least one data-flow and one control-flow) of where you did not have to use the maximum number of NOPs.

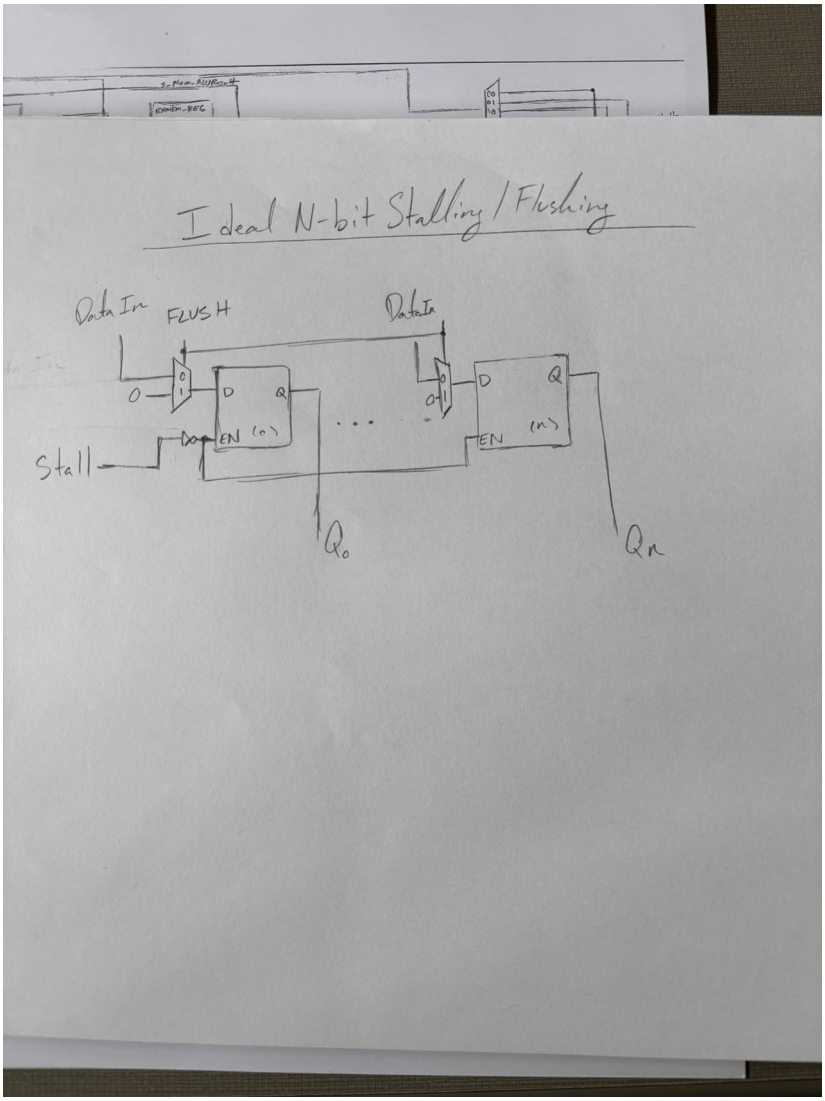
[1.d] report the maximum frequency your software-scheduled pipelined processor can run at and determine what your critical path is (specify each module/entity/component that this path goes through).

Maximum frequency: 51.04 MHZ

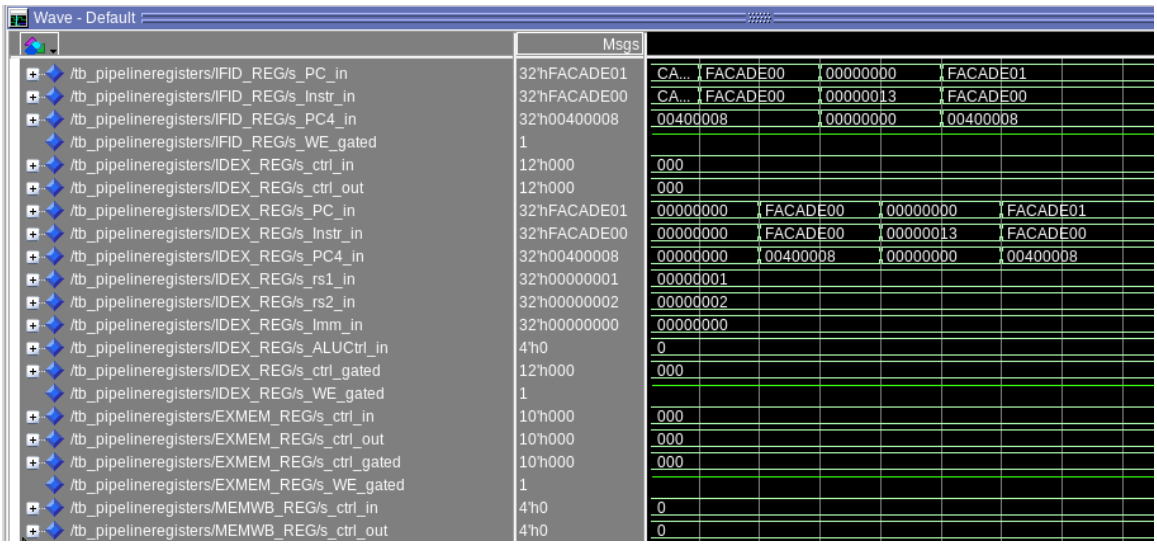
Critical Path: IDEX_REG > B_SRC_MUX > ALU > ALU output mux > branch_resolver > s_redirect > sw_fetch_unit (pc register)



[2.a.ii] Draw a simple schematic showing how you could implement stalling and flushing operations given an ideal N-bit register.



[2.a.iii] Create a testbench that instantiates all four of the registers in a single design. Show that values that are stored in the initial IF/ID register are available as expected four cycles later, and that new values can be inserted into the pipeline every single cycle. Most importantly, this testbench should also test that each pipeline register can be individually stalled or flushed.



[2.b.i] list which instructions produce values, and what signals (i.e., bus names) in the pipeline these correspond to.

Type	Instructions	Produced value	Pipeline signals
R	Add, sub, and, or, xor, slt, sltu, sll, srl, sra	Rd = ALU result	s_ALUOut s_MEM_ALU_Result s_WB_ALU_Result s_RegWrData

I	Addi, andi, ori, xori, slti, sltiu, slli, srli, srai	Rd = ALU result	s_ALUOut s_MEM_ALU_Result s_WB_ALU_Result s_RegWrData
Load	Lw, lh, lb, lhu, lbu	Rd = memory data	s_ALUOUt s_DmemOut s_LoadData s_WB_DmemOut s_RegWrData
JAL/JALR	Jal, jalr	Rd = PC + 4	s_EX_PC_Plus4 s_MEM_PC_Plus4 s_WB_PC_Plus4 s_RegWrData
LUI	Lui	Rd = Immediate	s_ALUOut s_MEM_ALU_Result s_WB_ALU_Result s_RegWrData
AUIPC	Auipc	Rd = PC + immediate	s_ALUOut s_MEM_ALU_Result s_WB_ALU_Result s_RegWrData
Store	Sw, sh, sb	mem[addr] = rs2	s_EX_rs2_fwd s_MEM_rs2_out s_DmemData s_DmemWr

[2.b.ii] List which of these same instructions consume values, and what signals in the pipeline these correspond to.

Type	Instructions	consumed value	Pipeline signals
R	Add, sub, and, or, xor, slt, sltu, sll, srl, sra	Rs1, rs2	s_rs1_data, s_rs2_data,

			s_EX_rs1_fwd, s_EX_rs2_fwd, s_ALU_A, s_B_ALU_choice
I	Addi, andi, ori, xori, slti, sltiu, slli, srli, srai	rs1	s_rs1_data, s_EX_rs1_fwd s_ALU_A
Load	Lw, lh, lb, lhu, lbu	rs1	s_rs1_data, s_EX_rs1_fwd, s_ALU_A, s_DMemAddr
JAL	Jal	xx	s_EX_PC
LUI	Lui	xx	xx
AUIPC	Auipc	Rd = PC + immediate	s_EX_PC
Store	Sw, sh, sb	rs1	s_rs1_data s_EX_rs1_fwd s_ALU_A s_DmemAddr s_rs2_data s_EX_rs2_fwd s_MEM_rs2_out s_DmemData
JALR	Jalr	rs1	s_rs1_data s_EX_rs1_fwd
Branch	Beq, bne, blt, bge, bltu, bgeu	Rs1, rs2	s_rs1_data s_rs2_data s_EX_rs1_fwd s_EX_rs2_fwd s_ALU_A s_B_ALU_choice

[2.b.iii] generalized list of potential data dependencies. From this generalized list, select those dependencies that can be forwarded (write down the corresponding pipeline stages that will be forwarding and receiving the data), and those dependencies that will require hazard stalls.

Producer	Consumer	Condition	Forwarding Possible?
EX/MEM	EX(rs1)	Mem_RegWrite = 1 AND Mem_rd != x0 AND MEM_rd = EX_rs1	YES s_MEM_ALU_Result > s_EX_rs1_fwd

EX/MEM	EX(rs2)	Mem_RegWrite = 1 AND Mem_rd != x0 AND MEM_rd = EX_rs2	YES s_MEM_ALU_Result > s_EX_rs2_fwd
EX/MEM	EX(store)	MEM_RegWrite = 1 AND MEM_rd != x0 AND MEM_rd = EX_rs2	YES s_MEM_ALU_Result > s_EX_rs2_fwd > s_MEM_rs2_out
MEM/WB	EX(rs1)	WB_RegWrite = 1 AND WB_rd != x0 AND WB_rd = EX_rs1 AND EX != MEM	YES s_RegWrData > s_EX_rs1_fwd
MEM/WB	EX(rs2)	WB_RegWrite = 1 AND WB_rd != x0 AND WB_rd=EX_rs2 AND EX != MEM	YES s_RegWrData > s_EX_rs2_fwd
WB	ID(rs1)	WB_RegWrite = 1 AND WB_rd != x0 AND WB_rd = ID_rs1	YES s_RegWrData > s_rs1_data
WB	ID(rs2)	WB_RegWrite = 1 AND WB_rd != x0 AND WB_rd = ID_rs2	YES s_RegWrData > s_rs2_data
EX/MEM	EX	EX_MEMRead = 1 AND EX_rd != x0 AND (EX_rd=ID_rs1 OR EX_rd=ID_rs2)	NO!

[2.b.iv] global list of the datapath values and control signals that are required during each pipeline stage

IF	PC Instr PC Plus 4
ID	PC Instr

	PC Plus 4
EX	PC
	Instr
	PC Plus 4
	rs1
	rs2
	immediate generated
	[CONTROLS]
	ALUSrc
	ALUCtrl
	isLUI
	isAUIPC
	Jump
	Branch
	PC SRC
	MemWrite
	MemRead
	RegWrite
	MemToReg
MEM	PC
	Instr
	PC Plus 4
	rs2
	ALU Result
	Jump
	Branch
	PC SRC
	MemWrite
	MemRead
	RegWrite
	MemToReg
WB	PC
	Instr
	PC Plus 4
	ALU Result
	DMemOut
	Rd Addr
	Halt

[2.c.i] list all instructions that may result in a non-sequential PC update and in which pipeline stage that update occurs.

Instruction	Pipeline Stage
Beq	EX
Bne	EX
Blt	EX
Bge	EX
Bltu	EX
Bgeu	EX
Jal	EX
Jalr	EX

[2.c.ii] For these instructions, list which stages need to be stalled and which stages need to be squashed/flushed relative to the stage each of these instructions is in.

Instruction	Stages to Stall	Stages to Flush
Beq		IF & ID
Bne		IF & ID
Blt		IF & ID
Bge		IF & ID
Bltu		IF & ID
Bgeu		IF & ID
Jal		IF & ID
Jalr		IF & ID

[2.d] implement the hardware-scheduled pipeline using only structural VHDL. As with the previous processors that you have implemented, start with a high-level schematic drawing of the interconnection between components.
I attached the schematic underneath the timing output stuff!

[2.e – i, ii, and iii] In your writeup, show the QuestaSim output for each of the following tests, and provide a discussion of result correctness. It may be helpful to also annotate the waveforms directly.

```
[shawnr@co1313-04 cpre3810-toolflow-3]$ ./3810_tf.sh test --summary proj/riscv/final_tests/*.s
Using VDI Python Environment
Testing
WARNING: Software tree location not set or invalid, using $MGC_HOME=/usr/local/mentor/calibre
None
All VHDL src files compiled successfully

 Assembly file                |  Rars  | Questasim | Test Passed |  CPI  | Results Directory
------------------------------+--------+-----------+-------------+-------+-----------------
 s/test_back_to_back_branch.s |  pass  |   pass    |    pass     |   2.0 | output/test_back_to_back_branch.s
 al_tests/test_beq_nottaken.s |  pass  |   pass    |    pass     |   2.0 | output/test_beq_nottaken.s
 final_tests/test_beq_taken.s |  pass  |   pass    |    pass     |   2.2 | output/test_beq_taken.s
 riscv/final_tests/test_bge.s |  pass  |   pass    |    pass     |   2.2 | output/test_bge.s
 iscv/final_tests/test_bgeu.s |  pass  |   pass    |    pass     |   2.2 | output/test_bgeu.s
 riscv/final_tests/test_blt.s |  pass  |   pass    |    pass     |   2.2 | output/test_blt.s
 iscv/final_tests/test_bltu.s |  pass  |   pass    |    pass     |   2.2 | output/test_bltu.s
 riscv/final_tests/test_bne.s |  pass  |   pass    |    pass     |   2.2 | output/test_bne.s
 final_tests/test_chain_fwd.s |  pass  |   pass    |    pass     |  1.67 | output/test_chain_fwd.s
 inal_tests/test_double_fwd.s |  pass  |   pass    |    pass     |   2.0 | output/test_double_fwd.s
 inal_tests/test_fwd_branch.s |  pass  |   pass    |    pass     |   2.2 | output/test_fwd_branch.s
 l_tests/test_fwd_exmem_rs1.s |  pass  |   pass    |    pass     |  2.33 | output/test_fwd_exmem_rs1.s
 l_tests/test_fwd_exmem_rs2.s |  pass  |   pass    |    pass     |  2.33 | output/test_fwd_exmem_rs2.s
 /final_tests/test_fwd_jalr.s |  pass  |   pass    |    pass     |   2.2 | output/test_fwd_jalr.s
 l_tests/test_fwd_memwb_rs1.s |  pass  |   pass    |    pass     |   2.0 | output/test_fwd_memwb_rs1.s
 l_tests/test_fwd_memwb_rs2.s |  pass  |   pass    |    pass     |   2.0 | output/test_fwd_memwb_rs2.s
 al_tests/test_fwd_priority.s |  pass  |   pass    |    pass     |   2.0 | output/test_fwd_priority.s
 inal_tests/test_fwd_wb_rs1.s |  pass  |   pass    |    pass     |   1.8 | output/test_fwd_wb_rs1.s
 inal_tests/test_fwd_wb_rs2.s |  pass  |   pass    |    pass     |   1.8 | output/test_fwd_wb_rs2.s
 riscv/final_tests/test_jal.s |  pass  |   pass    |    pass     |   3.0 | output/test_jal.s
 _tests/test_jal_jalr_combo.s |  pass  |   pass    |    pass     |  2.33 | output/test_jal_jalr_combo.s
 iscv/final_tests/test_jalr.s |  pass  |   pass    |    pass     |   2.2 | output/test_jalr.s
 v/final_tests/test_loaduse.s |  pass  |   pass    |    pass     |  1.83 | output/test_loaduse.s
 _tests/test_loaduse_branch.s |  pass  |   pass    |    pass     |   2.0 | output/test_loaduse_branch.s
 nal_tests/test_loaduse_fwd.s |  pass  |   pass    |    pass     |  1.83 | output/test_loaduse_fwd.s
 final_tests/test_store_fwd.s |  pass  |   pass    |    pass     |   1.8 | output/test_store_fwd.s
 inal_tests/test_x0_forward.s |  pass  |   pass    |    pass     |  2.33 | output/test_x0_forward.s
[shawnr@co1313-04 cpre3810-toolflow-3]$
```

[2.e.i] Create a spreadsheet to track these cases and justify the coverage of your testing approach. Include this spreadsheet in your report as a table.

Test #	Test Name	Hazard/Forward Type	Forward Stage	Receiving Stage	Pass Criteria
1	test_fwd_exmem_rs1	EX/MEM→EX forward rs1	EX/MEM	EX	x2=15
2	test_fwd_exmem_rs2	EX/MEM→EX forward rs2	EX/MEM	EX	x2=10
3	test_fwd_memwb_rs1	MEM/WB→EX forward rs1	MEM/WB	EX	x3=13
4	test_fwd_memwb_rs2	MEM/WB→EX forward rs2	MEM/WB	EX	x3=15
5	test_fwd_wb_rs1	WB→ID bypass rs1	WB	ID	x4=11
6	test_fwd_wb_rs2	WB→ID bypass rs2	WB	ID	x4=13
7	test_loaduse	Load-use stall	—	—	"x2=42
8	test_store_fwd	Store data forward	EX/MEM	EX (rs2)	x2=99
9	test_double_fwd	Simultaneous rs1+rs2 forward	EX/MEM+MEM/WB	EX	x3=15
10	test_fwd_priority	EX/MEM priority over MEM/WB	EX/MEM	EX	x2=20
11	test_x0_forward	x0 forward suppression	—	—	"x0=0
12	test_chain_fwd	Chained EX/MEM→EX forwards	EX/MEM	EX	x1-x5=1-5
13	test_loaduse_fwd	Load-use + forward combo	MEM/WB	EX	"x2=7
14	test_fwd_branch	Forward into branch operands	EX/MEM	EX/branch	"x3=0
15	test_fwd_jalr	Forward into JALR base	EX/MEM	EX/JALR	"x2=0
16	test_beq_taken	BEQ taken	—	—	"x3=0
17	test_beq_nottaken	BEQ not taken	—	—	x3=1
18	test_bne	BNE taken	—	—	"x3=0
19	test_blt	BLT taken	—	—	"x3=0
20	test_bge	BGE taken	—	—	"x3=0
21	test_bltu	BLTU taken	—	—	"x3=0
22	test_bgeu	BGEU taken	—	—	"x3=0
23	test_jal	JAL	—	—	"x2=0
24	test_jalr	JALR	—	—	"x2=0
25	test_branch_fwd_combo	Forward + branch simultaneously	EX/MEM	EX+branch	"x3=0
26	test_jal_jalr_combo	JAL followed by JALR	—	—	"x3=0
27	test_back_to_back_branch	Two branches in succession	—	—	"x5=0
28	test_loaduse_branch	Load-use stall + branch	—	—	"x3=0

[2.e.ii] Create a spreadsheet to track these cases and justify the coverage of your testing approach. Include this spreadsheet in your report as a table.

Test #	Test Name	Hazard/Forward Type	Forward Stage	Receiving Stage	Pass Criteria
1	test_fwd_exmem_rs1	EX/MEM→EX forward rs1	EX/MEM	EX	x2=15
2	test_fwd_exmem_rs2	EX/MEM→EX forward rs2	EX/MEM	EX	x2=10
3	test_fwd_memwb_rs1	MEM/WB→EX forward rs1	MEM/WB	EX	x3=13
4	test_fwd_memwb_rs2	MEM/WB→EX forward rs2	MEM/WB	EX	x3=15
5	test_fwd_wb_rs1	WB→ID bypass rs1	WB	ID	x4=11
6	test_fwd_wb_rs2	WB→ID bypass rs2	WB	ID	x4=13
7	test_loaduse	Load-use stall	—	—	"x2=42
8	test_store_fwd	Store data forward	EX/MEM	EX (rs2)	x2=99
9	test_double_fwd	Simultaneous rs1+rs2 forward	EX/MEM+MEM/WB	EX	x3=15
10	test_fwd_priority	EX/MEM priority over MEM/WB	EX/MEM	EX	x2=20
11	test_x0_forward	x0 forward suppression	—	—	"x0=0
12	test_chain_fwd	Chained EX/MEM→EX forwards	EX/MEM	EX	x1-x5=1-5
13	test_loaduse_fwd	Load-use + forward combo	MEM/WB	EX	"x2=7
14	test_fwd_branch	Forward into branch operands	EX/MEM	EX/branch	"x3=0
15	test_fwd_jalr	Forward into JALR base	EX/MEM	EX/JALR	"x2=0
16	test_beq_taken	BEQ taken	—	—	"x3=0
17	test_beq_nottaken	BEQ not taken	—	—	x3=1
18	test_bne	BNE taken	—	—	"x3=0
19	test_blt	BLT taken	—	—	"x3=0
20	test_bge	BGE taken	—	—	"x3=0
21	test_bltu	BLTU taken	—	—	"x3=0
22	test_bgeu	BGEU taken	—	—	"x3=0
23	test_jal	JAL	—	—	"x2=0
24	test_jalr	JALR	—	—	"x2=0
25	test_branch_fwd_combo	Forward + branch simultaneously	EX/MEM	EX+branch	"x3=0
26	test_jal_jalr_combo	JAL followed by JALR	—	—	"x3=0
27	test_back_to_back_branch	Two branches in succession	—	—	"x5=0
28	test_loaduse_branch	Load-use stall + branch	—	—	"x3=0

[2.f] report the maximum frequency your hardware-scheduled pipelined processor can run at and determine what your critical path is (specify each module/entity/component that this path goes through).

Maximum frequency: 41.12 MHZ
Critical Path: EX/MEM> FORWARDING > FWD_MUX_B > B_SRC_MUX > ALU > BRANCH_COND > IF/ID